

CertAlloc: Formally Verified, Deterministic Memory Allocator

Engineered for Safety-Critical and Hard Real-Time Systems (RTOS)

Author: Vindana Sandun | CyBurn Digital

Executive Summary

Dynamic memory allocation is strictly prohibited in safety-critical firmware (ISO 26262, DO-178C) due to non-deterministic execution times, heap fragmentation, and the catastrophic risk of out-of-memory hard faults.

CertAlloc is a freestanding, MISRA-C:2012 compliant memory pool allocator designed to bridge the gap between dynamic flexibility and static safety. By utilizing fixed-size memory pools and highly optimized bitwise free-list tracking, CertAlloc provides guaranteed **O(1) time complexity** with absolute zero memory fragmentation. The core engine and algorithmic state machine have been mathematically proven against memory leaks, double-frees, and deadlocks using industry-standard Bounded Model Checking (CBMC) and TLA+ temporal logic.

Architectural Guarantees

True O(1) Time Complexity

CertAlloc abandons traditional linked-list traversal. It utilizes single-cycle hardware intrinsic instructions coupled with a contiguous bitmapped indexing system to resolve memory blocks instantly.

- **Worst-Case Execution Time (WCET):** Bounded and mathematically predictable.
- **Allocation Latency:** ~190 CPU Cycles (x86_64 architecture).
- **Deallocation Latency:** ~152 CPU Cycles (x86_64 architecture).

Zero Heap Fragmentation

The allocator operates entirely on pre-partitioned, fixed-size memory slabs. Because blocks are never split or coalesced, internal and external fragmentation is fundamentally impossible by design. A maximum of 16 individual pool types are statically allocated at compile time.

Drop-In RTOS Integration

CertAlloc is designed as a seamless memory substrate for bare-metal applications or RTOS environments (FreeRTOS, Zephyr). Thread safety is achieved via weak concurrency hooks (`certalloc_enter_critical()` and `certalloc_exit_critical()`) that map directly to hardware interrupt-disable routines or system mutexes.

Formal Verification & Safety Proofs

CertAlloc is not just tested; its internal state machine has been formally verified. The architecture was subjected to rigorous mathematical proofs to guarantee the absence of undefined behavior across the entire operational state space.

1. Bounded Model Checking (CBMC)

The C implementation was verified using CBMC version 6.6.0. The SAT solver exhaustively evaluated the pointer arithmetic, boundary limits, and memory lifecycles, ultimately proving the instance is mathematically unsatisfiable for failure states.

Verification Target	Condition Checked	Result
Memory Isolation	Array upper bounds and pointer overflow/underflow	SUCCESS
Lifecycle Integrity	Double free detection	SUCCESS
Pointer Safety	Unaligned free detection	SUCCESS
Arithmetic Safety	Division by zero prevention	SUCCESS
Total Assertions	337 distinct logic gates evaluated	0 Failures

2. Algorithmic State Verification (TLA+)

To prove the core logic is incapable of deadlocking or losing synchronization, the allocation lifecycle was modeled in TLA+ and executed through the TLC2 Model Checker.

Verification Metric	State-Space Result
Distinct States Found	65,536
Total States Generated	1,048,577
State Graph Depth	17
Verification Status	0 errors found

Delivery & Integration Package

CertAlloc is packaged for immediate enterprise compilation, structured cleanly to separate production code from verification artifacts and benchmarking tools.

- `include/`: Public API header defining the static pool structures and concurrency hooks.
- `lib/`: Compiled static library (`libcertalloc.a`) for direct linkage.
- `tools/`: Compiled static analyzer binary for validating pool configurations.
- `compliance/`: Formal proof logs containing the raw TLC output and CBMC solver steps.
- `benchmark/`: Cycle-accurate hardware benchmarking routines.